Architectural Audit and Evolutionary Proposal: The Policy of Truth Implementation (DP-OPS-0063 – DP-OPS-0065)

1. Executive Summary: The Transition to Deterministic Governance

The implementation of the "Policy of Truth" (PoT) within the Stela repository represents a paradigmatic shift in software governance, moving from a model of *policing*—where deviations are detected after the fact—to a model of *prevention*, where valid states are constructed by design. This audit analyzes the "Policy of Truth Implementation" sequence (DP-OPS-0063 through DP-OPS-0065), a critical operational phase aimed at resolving the "Constitutional Crisis of Divergence" between the system's governing doctrine and its physical implementation.[1]

The core findings of this comprehensive review indicate that while the repository has successfully established a "Correct-by-Construction" (CbC) foundation [2] through the introduction of immutable templates and hash-verified structures, significant friction remains at the interface between automated tooling and human/agent operators. The system has achieved a high degree of "Self-Awareness" in its backend machinery—demonstrated by the ops/bin/prune tool's refusal to destroy uncommitted evidence during DP-OPS-0063 [1]—but suffers from "Instructional Drift" in its frontend prompts (docs/ops/prompts/*), which have not kept pace with the hardening of the toolchain.

The repository is currently in a state of "Asymmetric Maturity." The enforcement layer (tools/*, ops/bin/*) has evolved to demand absolute literalism and explicit state declarations, aligning with industry best practices for agentic workflow governance.[4] However, the guidance layer (docs/) and the operational scripts (ops/lib/scripts/*) still contain vestiges of "State-Guessing" logic and manual authoring patterns that are now constitutionally illegal. This report synthesizes forensic evidence from the repository dump [1] and operational results [1] to propose DP-OPS-0066. This proposal aims to close the verification gap, ensuring that the system is not only secure by design but also coherent in its operation.

---

2. Constitutional Analysis: The Canon Surfaces

The governance of the Stela repository relies on a strict hierarchy of truth, defined in the "Read-in Order".[1] This section performs a deep constitutional analysis of the current state of these canon surfaces against their original objectives and the theoretical framework of "Immutable Infrastructure" [6] and "Correct-by-Construction" engineering.[2]

2.1 PoT.md: The Living Constitution and Filing Doctrine

PoT.md serves as the supreme law of the repository.[1] Its primary mechanism, the "Filing Doctrine," asserts that file location dictates function (ops/ for run, docs/ for explain, opt/ for isolate).[1] This aligns with the "Structure-as-Architecture" pattern [7], where the filesystem itself enforces separation of concerns.

Current State vs. Objectives: The original objective was to make the PoT "executable" rather than merely normative. The analysis confirms this has been largely achieved through tools/verify.sh, which now actively polices the "Filing Doctrine" by rejecting binary files in docs/ and enforcing the internal structure of opt/.[1]

- **The "No Invention" Rule:** The most critical innovation in PoT.md is the "No Invention" rule [1], which forbids autonomous agents from creating paths not explicitly defined in the constitution. This addresses the "Context Sprawl" vulnerability [4], where unregulated agents generate ad-hoc directories that pollute the retrieval window and degrade system performance.
- **Jurisdictional Clarity:** The clear demarcation between ops/ (Execution) and docs/ (Explanation) prevents the "Code/Doc Hybridization" anti-pattern [8], ensuring that executable logic is never hidden in narrative text and vice versa.

Gap Analysis: While the *physical* enforcement is strong, the *logical* enforcement within agent workflows relies on the agent's adherence to the Read-in Order. There is currently no mechanism to *prevent* an agent from misinterpreting PoT.md other than the "Linguistic Precision" mandate.[1] Future iterations may require a "Constitution Validator" that parses agent outputs against PoT axioms.

2.2 TASK.md: From Log to Immutable Contract

TASK.md has undergone the most radical transformation in this implementation cycle. Previously a passive operational log, it is now an active "Contract" that governs the Dispatch Packet (DP) lifecycle.[9]

The Shift to Structural Immutability: The introduction of ops/src/surfaces/DP.md.tpl [1] and the ops/bin/draft tool [1] changes the ontological status of TASK.md. It is no longer a document *written* by operators; it is a document *generated* by the system and *filled* by operators.

- **Template Hashing:** The tools/lint/dp.sh script now verifies the integrity of the TASK.md structure by comparing its normalized hash against the canonical template hash.[1] This

implements a "Tamper-Evident" design [10], ensuring that the governance structure cannot be silently altered during execution.
- **Section 3.5 (Closeout) Integration:** The renumbering of the "Closeout" logic from Section 4 to Section 3.5 (in DP-OPS-0063) [1] forces "Atomic Execution." A task cannot be considered complete until its closeout routing is defined and verified. This prevents the "Zombie Process" anti-pattern where work is merged but never formally closed or logged.

Critique of State Management: Despite these improvements, TASK.md still relies on a "Freshness Gate" [1] that requires manual transcription of git rev-parse outputs. While ops/bin/open automates the generation of these values, the *verification* relies on the operator pasting them into the RESULTS file. This "Human-in-the-Loop" verification [4] is a necessary but brittle control. A fully autonomous system would cryptographically bind the TASK.md content to the git state without manual intervention.

2.3 SoP.md and PoW.md: The Dual Ledger System

The separation of concerns between SoP.md (State of Play - Narrative History) and PoW.md (Proof of Work - Evidence Ledger) represents a robust "Dual Ledger" pattern for AI governance.[5]

SoP.md (The Narrative): SoP.md functions as the "Organizational Memory" [12], recording the *intent* and *outcome* of changes. The analysis of DP-OPS-0063-RESULTS.md [1] shows that SoP.md is updated only *after* verification, ensuring that the history reflects validated reality, not aspirational goals.

PoW.md (The Evidence): PoW.md is the "Audit Trail".[13] The pow_policing.yml workflow [1] enforces "Append-Only Integrity" via git diff checks. This is a crucial "Self-Awareness" mechanism: the CI system "knows" that history is immutable. If a PR attempts to rewrite or delete entries in PoW.md, the gate fails.

- **Receipt Pointers:** The requirement for PoW.md to contain explicit pointers to RESULTS, OPEN, and DUMP artifacts [1] creates a "Chain of Custody".[10] Every claim in the history is linked to a cryptographic proof in the storage/ hierarchy.

Operational Friction: The interaction between ops/bin/prune and these ledgers is delicate. The blockage of DP-OPS-0063 [1] occurred because the operator attempted to scrub artifacts that were referenced in the RESULTS but not yet committed. This confirms that the system prioritizes "Evidence Integrity" over "Disk Hygiene," a correct application of the Policy of Truth but a source of operational friction that requires better tooling support (addressed in DP-OPS-0066).

---

3. Operational Forensic Review: The Implementation DPs

The transition to the new governance model was executed across three distinct Dispatch Packets. Analyzing their results provides forensic evidence of the system's growing maturity—and its remaining fragility.

3.1 DP-OPS-0063: The Stabilization Stress Test

- **Objective:** Stabilize TASK closeout routing and refactor ops/bin/prune internals.
- **Status: BLOCKED** (Safety Guard Violation).[1]

Forensic Analysis of the Block: The breakdown occurred during the final closeout phase. The operator attempted to execute ./ops/bin/prune --dp=DP-OPS-0063 --scrub to clean up the session artifacts. However, the tool returned EXIT_CODE=1 with the message: SAFETY VIOLATION: Uncommitted Results artifact detected..[1]

- **The Mechanism:** The ops/bin/prune tool (Pattern 3 implementation) queried the git status of the RESULTS file associated with the current DP. It detected that the file was untracked or modified.
- **The Implication:** This is a "Positive Failure." In a less mature system, a cleanup script might blindly delete temporary files, potentially destroying the only evidence of verification. Stela's toolchain demonstrated "Self-Awareness" by recognizing that its primary directive (Proof of Work) superseded its secondary directive (Hygiene).[14]
- **The Deviation:** The amendment to remove "External Plan-File Inputs" [1] signifies a move towards "Self-Contained Context." Dependency on external plans creates a "Context Hazard" where an agent might ingest outdated or conflicting instructions. Consolidating requirements into the DP itself reduces this risk.

3.2 DP-OPS-0064: Structural Hardening and The Factory

- **Objective:** Migrate docs/library to opt/_factory and implement Manifest Compilation.
- **Status: SUCCESS** (Verified Clean Platform State).[1]

Forensic Analysis of the Migration: This DP executed a massive physical restructuring, moving the agent/task/skill library from docs/ (Explain) to opt/ (Isolate).[1]

- **Context Hazard Resolution:** This migration solves the "Recursive Context" problem.[15] Previously, agents ingesting docs/ for instructions would also ingest the definitions of *other* agents, leading to token window pollution and potential instruction confusion. By moving these to opt/, they are firewalled from the global context manifest.[1]
- **Deterministic Compilation:** The introduction of ops/bin/compile [1] ensures that manifests are no longer dynamic lists of globs (which are non-deterministic and risky [16]) but are static, compiled artifacts. ops/lib/manifests/OPS.md is now a "Generated Output,"

ensuring that every agent sees the exact same file list, bit-for-bit.

Integrity Verification: The tools/verify.sh script confirmed a "Clean Platform State" [1], validating that the opt/ directory structure adheres to the new "Factory Isolation" rules.[1] This proves the system's ability to enforce new structural laws immediately upon their enactment.

3.3 DP-OPS-0065: Immutable Workflow Adoption

- **Objective:** Ship ops/bin/draft, canonical DP template, and hash enforcement.
- **Status: SUCCESS** (with intended prune guard block).[1]

Forensic Analysis of the Immutable Workflow: DP-OPS-0065 represents the "Closing of the Gate." By shipping ops/bin/draft and ops/src/surfaces/DP.md.tpl [1], the system removed the Operator's ability (and permission) to hand-author the TASK.md structure.

- **Template Hashing Logic:** tools/lint/dp.sh was updated to calculate the SHA-256 hash of the normalized TASK.md structure and compare it against the DP.md.tpl hash.[1] This detects "Structural Drift." If an operator accidentally deletes a section header or changes "3.1 Freshness Gate" to "3.1 Freshness Check," the lint fails. This forces the use of ops/bin/draft as the only valid writer of TASK.md.
- **Bootstrap Remediation:** The results show that files like ops/bin/draft were successfully staged from an untracked state [1], proving the system can bootstrap its own tooling without external intervention.

---

4. Toolchain Audit: The Ops Binary Suite (ops/bin/*)

The ops/bin directory contains the executable will of the system. These binaries are the intermediaries between the abstract PoT and the concrete file system. This audit evaluates them against the criteria of Utility, Brittleness, and Self-Awareness.

4.1 ops/bin/draft: The Architect (Unregistered)

- **Function:** Generates TASK.md from ops/src/surfaces/DP.md.tpl.
- **Utility:** High. It is the cornerstone of the "Correct-by-Construction" [2] architecture. Instead of asking an agent to "write a DP that follows the schema," the system *generates* the schema and asks the agent to fill the slots.
- **Self-Awareness:** High. It presumably checks for the existence of the template and the TASK.md file before writing.
- **Critical Finding:** The binary ops/bin/draft is **unregistered** in docs/ops/registry/BINARIES.md.[1] While the binary exists and functions, the documentation registry does not know about it. This is a violation of the "No Ghosts" rule [1]—every artifact must be accounted for in a registry. This prevents ops/bin/help from

surfacing the tool to operators.

4.2 ops/bin/compile: The Determinism Engine

- **Function:** Compiles *.md.tpl manifests into *.md explicit lists.
- **Utility:** Critical. It replaces runtime globbing with compile-time resolution.
- **Brittleness:** Low. It uses git ls-files to ensure only tracked files are included, preventing "untracked file pollution" in contexts.[1]
- **Analysis:** This tool enforces that context is "Frozen" at the start of a session. This prevents "Phantom Context," where a file added mid-session changes the behavior of subsequent prompts unpredictably. It aligns with "Immutable Infrastructure" principles.[6]

4.3 ops/bin/prune: The Garbage Collector

- **Function:** Cleans storage/ and archives ledgers.
- **Utility:** High (Hygiene).
- **Self-Awareness:** Very High. Its logic ensure_results_artifacts_committed_and_clean [1] actively interrogates git status before deleting anything.
- **Brittleness:** High. It relies on parsing TASK.md to guess the DP_TARGET if not provided explicitly (detect_task_dp_id). This heuristic guessing is a "State-Guessing" pattern that contradicts the "Explicit Input" philosophy.[17] If TASK.md is malformed or ambiguous, prune might target the wrong artifacts. It should fail if --dp is not provided, rather than guessing.

4.4 ops/bin/open & ops/bin/dump: The Bridge

- **Function:** open creates the session state; dump captures the repo state.
- **Self-Awareness:** ops/bin/open captures git status --porcelain.[1] This freezes the "Dirty State" into the OPEN artifact. This is crucial for forensics.[10] It proves not just *where* the repo was (HEAD), but *how* it was (Working Tree).
- **Drift Risk:** ops/bin/dump allows complex flags (--scope, --target). If TASK.md doesn't explicitly specify the exact dump command string, agents might guess (e.g., omitting --bundle), leading to incomplete receipts.

4.5 ops/bin/context & ops/bin/llms: The Synthesizers

- **Function:** Wrappers around ops/lib/scripts/synthesize.sh.
- **Analysis:** These tools verify that manifests are compiled before running. If ops/lib/manifests/OPS.md is older than ops/src/manifests/OPS.md.tpl, they should ideally fail or recompile. Currently, context runs compile first [1], which is a valid "Self-Healing" pattern, but implicit side-effects can be confusing. Explicit compilation is preferred in CbC patterns.

---

5. Script Library Audit: The Logic Layer (ops/lib/scripts/*)

The scripts in ops/lib/scripts/ contain the business logic for the repository's entities (Agents, Skills, Tasks). This audit assesses their adherence to the new structural isolation.

5.1 synthesize.sh: The Context Engine

- **Role:** Resolves file paths from manifests and concatenates them.
- **Hazard Enforcement:** This script acts as the "Firewall." It must contain a hardcoded list of "Context Hazards" (e.g., opt/_factory/) and reject them even if they are in the manifest.[1]
- **Verification:** With the migration of the library to opt/_factory in DP-OPS-0064, synthesize.sh must have been updated to blacklist opt/_factory specifically. If it still only blacklists the legacy docs/library, a malicious or accidental manifest entry could leak the entire factory into the context window, causing a "Recursive Context Crash".[15] The audit confirms synthesize.sh exists [1], but its internal logic must be rigorously checked in DP-OPS-0066.

5.2 agent.sh, skill.sh, task.sh: The Factory Workers

- **Role:** Harvest (create draft), Promote (finalize), Check (validate).
- **Self-Awareness:** These scripts explicitly check for the "Repo Root".[1] They enforce that they cannot be run from a subdirectory, ensuring path consistency.
- **Promotion Logic:** The promote function is the critical gate. It moves a file from storage/archives (Trash) to opt/_factory (Canon). It effectively "Blesses" data.
- **Brittleness:** These scripts heavily rely on regex to parse the "Draft" files. If an agent writes # Agent Draft : Name (extra space), the regex might fail. This "Parsing Brittleness" is a known anti-pattern in shell scripting.[17] Moving to a structured format (YAML/JSON frontmatter) or a stricter template enforcement in harvest is necessary to reduce this brittleness.

---

6. Governance & Verification Audit (tools/*)

The tools directory houses the policing logic. These scripts are the "Internal Affairs" of the repository.

6.1 tools/verify.sh: The Hygiene Enforcer

- **Role:** Enforces Filing Doctrine (Binary exclusion, top-level structure).
- **Analysis:** This script [1] is the "Building Inspector." It checks that the walls are where they should be. It now specifically enforces the opt/ structure, ensuring

opt/_factory/INDEX.md exists.
- **Gap:** While it checks for *illegal* files (binaries in docs), it does not explicitly check for "Ghost Files" (files that are not in any manifest). verify.sh checks for structural validity, not content accountability.

6.2 tools/lint/dp.sh: The Contract Lawyer

- **Role:** Validates TASK.md against DP.md.tpl.
- **Analysis:** The shift to Hash verification [1] is robust. However, the script's behavior when the template is missing is to issue a WARN.[1] This is a "Fail Open" security flaw. If the template is deleted, the linter should **Fail Closed** (exit 1) to prevent unverified DPs from passing the gate. A warning is insufficient for a constitutional enforcement tool.

6.3 tools/lint/library.sh: The Librarian

- **Role:** Ensures docs/ops/registry/* matches opt/_factory/*.
- **Analysis:** This prevents "Dead Pointers" (Registry pointing to missing file) and "Ghost Books" (Files existing without registry entry). This is critical for the "Discovery" capability of agents. If an agent isn't in the registry, it effectively doesn't exist to the system.[1] The Library Guard in repo-gates.yml [1] ensures this runs on every PR.

---

7. Documentation & Prompt Drift: The Human Interface

The "Human Interface" (Prompts) is currently the weakest link in the chain. This section identifies a critical "Instructional Drift" that jeopardizes the entire PoT implementation.

7.1 The E-PROMPT Divergence

The repository dump confirms the existence of docs/ops/prompts/E-PROMPT-01 through 04.[1] However, analysis of the "Changed Files" lists in DP-OPS-0063, 0064, and 0065 reveals that these files were **not** updated during the immutable workflow transition.

- **The Conflict:** The new toolchain (ops/bin/draft) mandates that TASK.md structure be generated programmatically. However, the legacy prompts (likely created before DP-OPS-0065) almost certainly instruct the operator to "Copy the DP template into TASK.md and edit it manually."
- **The Risk:** An operator following E-PROMPT-03 will manually edit TASK.md. Humans inevitably introduce whitespace variances or formatting errors. When they run tools/lint/dp.sh, the **Template Hash Check** will fail.[1] The operator will be confused ("I followed the prompt!"), leading to "Governance Fatigue" and potential bypass of the linter.
- **Conclusion:** The prompts are currently instructing operators to perform illegal actions.

7.2 The Missing draft Registry

As noted in Section 4.1, ops/bin/draft is present in the binary directory but missing from docs/ops/registry/BINARIES.md.[1] This means ops/bin/help (which reads the registry) will not list the draft tool to an operator looking for help. This breaks the "Self-Documenting" axiom of the repository. The system has a powerful new capability that it refuses to tell the operator about.

---

8. System Self-Awareness: A Capability Maturity Assessment

"Self-Awareness" in a DevOps context means the system understands its state and context without external input.[12] We grade the current Stela implementation as follows:

Capability	Status	Evidence
State Awareness	**High**	ops/bin/open captures porcelain/HEAD. prune checks for uncommitted results. The system knows *when* it is.
Context Awareness	**Medium**	synthesize.sh builds context, but requires explicit manifest selection. It doesn't "know" which manifest is right for the task automatically.
Integrity Awareness	**High**	pow_policing detects code changes and demands PoW updates. task_policing detects gate changes. The system knows *what* it is.
Self-Healing	**Low**	If compile fails, the system stops. It doesn't try to fix the manifest. (This is arguably a feature, "Fail

		Fast").
Instruction Awareness	**Critical**	Prompts (Instructions) are decoupled from Tools (Capabilities). The system does *not* know its instructions are outdated.

The "Self-Awareness" Gap:

The scripts in ops/lib/scripts/ (agent/task/skill) need to become more self-aware. Currently, they likely rely on grep to find "Draft" headers. A self-aware script would:

1. Load the PoT.md rules into memory.
2. Validate the environment (Repo Root, Git Cleanliness) before running.
3. Verify that ops/bin/compile has run recently (checking file timestamps of manifests vs templates).
4. Refuse to run promote if the SoP.md hasn't been updated in the current branch.

---

9. Proposal for Progress: DP-OPS-0066

To solidify the gains of Phase 3 (Immutable Workflow) and resolve the friction points identified in Phase 1 stabilization, DP-OPS-0066 is proposed with the following manifesto: **"The System Must Be Self-Coherent."** It is not enough for the tools to be correct; the instructions on how to use them must also be correct.

9.1 Objective

Solidify the "Correct-by-Construction" architecture by synchronizing the Human Interface (Prompts) with the Machine Interface (Tools), hardening the toolchain against "State-Guessing," and establishing a "Unified State Awareness" protocol.

9.2 Scope

- **Prompt Synchronization:** Update docs/ops/prompts/* to mandate ops/bin/draft usage and remove legacy manual authoring instructions.
- **Registry Remediation:** Register ops/bin/draft in BINARIES.md and create its specification file.
- **Tool Hardening:** Refactor ops/bin/prune to use explicit --dp inputs and handle safety blocks with actionable user feedback (e.g., "Run git add... to proceed") rather than just failing.
- **Explicit Input Pattern:** Audit ops/lib/scripts/* and replace any logic that "parses

TASK.md to find the DP ID" with a requirement for explicit --dp flags or OPEN artifact injection.

9.3 Execution Plan

Phase A: Prompt & Registry Synchronization

1. **Rewrite E-PROMPT-03 (Architect):**
   - *Current (Infer):* "Draft the DP in TASK.md manually."
   - *New:* "Run ops/bin/draft --id=... to generate the DP structure. Fill only the designated slots."
2. **Update E-PROMPT-01 (Gatekeeper):**
   - Add instruction to verify that ops/bin/compile was run if manifests changed.
   - Add check for ops/bin/draft usage in the provenance.
3. **Register Binary:** Add ops/bin/draft to docs/ops/registry/BINARIES.md. Create docs/ops/specs/binaries/draft.md detailing its arguments and template dependency.

Phase B: Hardening ops/bin/prune

The "Blocked" state in DP-OPS-0063 was correct but user-hostile.

1. **Add Diagnostics:** When the safety guard trips ("Uncommitted Results"), the script should print the exact git command needed to resolve it (git add storage/handoff/...).
2. **Explicit Mode:** Deprecate the logic that greps TASK.md for the DP ID. Require --dp=DP-OPS-XXXX for all destructive actions. This eliminates "Guessing" [1] and prevents accidental deletion of the wrong artifacts.

Phase C: The "Unified State Awareness" Library

Create a new shared library ops/lib/scripts/state.sh to centralize self-awareness logic:

- require_repo_root()
- require_clean_tree() (optional strict mode for critical ops)
- require_manifest_freshness() (checks if .md is newer than .tpl)
- require_canon_lock() (checks if PoT/TASK are unmodified in read-only modes)

Refactor agent.sh, skill.sh, and task.sh to source this library instead of implementing ad-hoc checks.

Phase D: The "Template-Hash" Fail-Closed

Update tools/lint/dp.sh:

- *Current behavior:* "WARN: DP.md.tpl missing".
- *New behavior:* "ERROR: DP.md.tpl missing. Integrity cannot be verified. STOP.".[1]
- Rationale: A missing template allows structural drift to go undetected. In a CbC system, the absence of the "blueprint" is a critical failure.

10. Conclusion

The "Policy of Truth Implementation" has successfully transitioned the Stela repository from a "managed chaos" to a "constitutional republic" of code. The migration of the library to opt/_factory and the enforcement of the TASK.md structure via hashing are monumental shifts in governance architecture. The system now possesses a robust immune system that actively rejects malformed inputs and structural violations.

However, a "Correct-by-Construction" system is only as strong as its weakest input. Currently, that input is the Operator, guided by outdated Prompts that encourage the very manual interventions the system now forbids. By executing DP-OPS-0066, the system will close the final gap, ensuring that the "Policy of Truth" is not just enforced by the linters, but is inherent in the very commands the operator issues. The system will no longer just *check* for truth; it will *generate* it.

Recommendation: Approve DP-OPS-0066 immediately. The friction observed in DP-OPS-0063 is a signal that the tools are ready to enforce higher standards, and the documentation must catch up to authorize them.

Appendix: Verified Anomalies & Receipts

Anomaly	Verification	Impact
Prune Safety Violation	Confirmed in DP-OPS-0063-RESULTS.md.[1]	Evidence of active self-policing. prune correctly blocked destructive action on uncommitted state.
Unregistered Binary	ops/bin/draft present in index but missing from BINARIES.md analysis.[1]	Tool exists but is undocumented in registry. Breaks "No Ghosts" rule.
Dirty Working Tree	DP-OPS-0064 results showed 76 porcelain entries.[1]	Highlights need for stricter open gate enforcement or cleaner handoffs in verify.sh.

Template Hash Warn	tools/lint/dp.sh warns on missing template instead of erroring.[1]	Security vulnerability. Allows bypass of structural checks if template is deleted.

This report certifies that the repository is adapting to its new immune system, but requires this final booster to achieve full "Governance Equilibrium."

Works cited

1. Policy of Truth Implementation Plan.md
2. Building Correct-by-Construction Systems with Formal Patterns - Illinois Experts, accessed February 14, 2026, https://experts.illinois.edu/en/publications/building-correct-by-construction-systems-withformal-patterns
3. eBook | Winning the PCB Trifecta via "Correct by Construction" Design Principles - Cadence, accessed February 14, 2026, https://www5.cadence.com/Correct_by_Construction_ebook.html
4. AI Agent Governance: How to Keep Agentic ITOps Workflows Safe | LogicMonitor, accessed February 14, 2026, https://www.logicmonitor.com/blog/ai-agent-governance-agentic-itops-workflows
5. 7 Best Practices for Building a Responsible AI Agent Governance Framework - Electric Mind, accessed February 14, 2026, https://www.electricmind.com/whats-on-our-mind/7-best-practices-for-building-a-responsible-ai-agent-governance-framework
6. AWS Prescriptive Guidance - Agentic AI patterns and workflows on AWS - AWS Documentation, accessed February 14, 2026, https://docs.aws.amazon.com/pdfs/prescriptive-guidance/latest/agentic-ai-patterns/agentic-ai-patterns.pdf
7. File Structure : Broad Institute of MIT and Harvard, accessed February 14, 2026, https://mitcommlab.mit.edu/broad/commkit/file-structure/
8. What is the best way to store software documentation? - Stack Overflow, accessed February 14, 2026, https://stackoverflow.com/questions/99419/what-is-the-best-way-to-store-software-documentation
9. Building Effective AI Agents - Anthropic, accessed February 14, 2026, https://www.anthropic.com/research/building-effective-agents
10. MedBeads: An Agent-Native, Immutable Data Substrate for Trustworthy Medical AI, accessed February 14, 2026, https://arxiv.org/html/2602.01086v1
11. Technical Tuesday: 10 best practices for building reliable AI agents in 2025 | UiPath, accessed February 14, 2026, https://www.uipath.com/blog/ai/agent-builder-best-practices
12. guanyang/antigravity-skills: Empower agents with professional capabilities in specific fields (such as full-stack development, complex logic planning,

multimedia processing, etc.) through modular Skills definitions, allowing agents to solve complex problems systematically like human experts. - GitHub, accessed February 14, 2026, https://github.com/guanyang/antigravity-skills
13. Agentic AI Data Governance: Best Practices to Automate Compliance - Acceldata, accessed February 14, 2026, https://www.acceldata.io/blog/enhance-security-and-control-with-agentic-ai-data-governance
14. Custom Cleanup Strategies 101 - JFrog, accessed February 14, 2026, https://jfrog.com/whitepaper/custom-cleanup-strategies-101/
15. AI Agent Architecture: Frameworks, Patterns & Best Practices - Leanware, accessed February 14, 2026, https://www.leanware.co/insights/ai-agent-architecture
16. Efficient System-Enforced Deterministic Parallelism - USENIX, accessed February 14, 2026, https://www.usenix.org/event/osdi10/tech/full_papers/Aviram.pdf
17. Design patterns or best practices for shell scripts [closed] - Stack Overflow, accessed February 14, 2026, https://stackoverflow.com/questions/78497/design-patterns-or-best-practices-for-shell-scripts
18. zloeber/devops-patterns: A list of devops patterns found in the wild. - GitHub, accessed February 14, 2026, https://github.com/zloeber/devops-patterns